

COP2034C Module 4 Programming Assignment

Instructions

In this assignment, we will expand the Palmer Penguins program from Module 3 by adding **decision-making** to the program. In Module 3, we organized the Palmer Penguins data into lists and used Python functions and list methods to calculate statistics. In this module, we will allow the user to select one of the penguin species and use **branches** to determine what the program should do based on that selection.

We will practice:

- **if**, **elif**, and **else** statements,
- equality and relational operators,
- detecting ranges,
- logical operators,
- membership operators,
- nested branches,
- and a conditional expression.

We will continue working incrementally: add a small amount of code, run the program, verify that it works correctly, and then continue.

Development Environment

This assignment will be completed in Google Colab, beginning with the Palmer Penguins program completed for Module 3.

After your program executes correctly:

1. Download your notebook as a Python (.py) file.
2. Upload the .py file to your forked GitHub repository.
3. Take a screenshot showing the successful execution of your program in Google Colab.
4. Save the screenshot as a .png or .jpg file.
5. Upload the screenshot to your GitHub repository.

Do not submit the .ipynb notebook.

1. Start with Your Module 3 Solution

Open the Palmer Penguins notebook you completed for Module 3. Rename the notebook for Module 4. A suggested name is:

PalmerPenguinsM4.ipynb

The Python file you eventually download should have a meaningful name such as:

PalmerPenguinsM4.py

Do not rewrite the program from the beginning. Module 4 should build on your completed Module 3 program.

2. Update the ID Header

Update the four-line ID header at the top of your program:

```
# PalmerPenguinsM4.py  
# Your Name  
# Submission Date  
# Uses branches to explore Palmer Penguin statistics.
```

Leave a blank line after the header.

3. Retain Your Module 3 Code

Keep the constants, lists, calculations, and formatted output from Module 3. Your program should still contain the three species names:

```
SP_CHINSTRAP = "Chinstrap"  
SP_GENTOO = "Gentoo"  
SP_ADELIE = "Adelie"
```

and their observation counts:

```
NUM_CHINSTRAP = 68  
NUM_GENTOO = 123  
NUM_ADELIE = 151
```

Continue using the parallel lists created in Module 3:

```
species_names = [  
    SP_CHINSTRAP,  
    SP_GENTOO,  
    SP_ADELIE  
]  
  
species_counts = [  
    NUM_CHINSTRAP,  
    NUM_GENTOO,  
    NUM_ADELIE  
]
```

Your existing Module 3 calculations for the total, average, percentages, largest species, and smallest species should remain in the program.

4. Create a List of Species Percentages

In Module 3, the percentage represented by each species was stored in three variables:

```
chinstrap_percent
gentoo_percent
adelie_percent
```

For this assignment, after calculating those values, create another list named `species_percentages`:

```
species_percentages = [
    chinstrap_percent,
    gentoo_percent,
    adelie_percent
]
```

This creates a third parallel list. The same index now refers to the same species in all three lists. For example:

```
species_names[1]    Gentoo
species_counts[1]   123
species_percentages[1] 35.96...
```

This will allow us to select a species and then retrieve all of its related information using the same index.

5. Add an Interactive Species Selection Section

After the existing Module 3 output, add a new section that allows the user to explore one species. Display a heading such as:

Explore a species:

Then ask the user to enter one of the three species names. For example (user input shown in red):

Enter Chinstrap, Gentoo, or Adelie: **Gentoo**

Store the input in a variable named:

```
species_choice
```

For this assignment, the user should enter the species name exactly as displayed. Python string comparisons are case-sensitive, so `Gentoo` and `gentoo` are not considered equal.

6. Validate the Species Using a Membership Operator

Before trying to process the user's selection, determine whether `species_choice` appears in the `species_names` list. Use the **not in** membership operator:

if species_choice not in species_names:

If the species is not valid, display a message such as:

Emperor is not one of the species in this dataset.

Otherwise, use an **else** branch to continue processing the valid selection.

All of the remaining species-selection code described below should execute only when the user enters a valid species.

7. Determine the Selected Species Index Using if, elif, and else

Inside the valid-selection branch, determine which species the user entered. Use a multi-branch **if-elif-else** statement and the equality operator **==**. For example, begin with:

```
if species_choice == species_names[0]:  
    selected_index = 0
```

Use an **elif** branch to check the second species.

Because the outer membership test has already established that the user's entry is valid, the final **else** branch can assign the index of the remaining species. For this step, use the **if-elif-else** structure rather than the **index()** list method. The purpose of this part of the assignment is to practice branching and equality comparisons.

8. Retrieve the Selected Species Data

After determining `selected_index`, retrieve the related values from the three parallel lists:

```
selected_species = species_names[selected_index]  
selected_count = species_counts[selected_index]  
selected_percent = species_percentages[selected_index]
```

Because the lists are parallel, the same index retrieves the name, count, and percentage belonging to the selected species.

9. Classify the Species by Its Percentage

Next, use a multi-branch **if-elif-else** statement to classify the selected species according to the percentage of the dataset that it represents. Use these assignment categories:

Percentage of dataset Category

Less than 25%	small
25% to less than 40%	medium
40% or greater	large

Create a variable named:

size_category

Begin with:

```
if selected_percent < 25:  
    size_category = "small"
```

For the medium category, use the logical **and** operator to explicitly identify both ends of the range:

```
elif (selected_percent >= 25) and (selected_percent < 40):
```

Use the final **else** branch for percentages of 40% or greater. With the Palmer Penguins data:

- Chinstrap should be classified as **small**.
- Gentoo should be classified as **medium**.
- Adelie should be classified as **large**.

10. Determine the Species' Relative Position

We already identified the species with the most and fewest observations in Module 3. Use the logical **or** operator to determine whether the selected species is one of those two groups:

```
if (selected_species == largest_species) or \  
    (selected_species == smallest_species):
```

Inside this branch, use a **nested if-else statement** to determine whether the selected species is specifically the largest or the smallest. Create a variable named:

position_description

Its value should become one of the following:

```
largest  
smallest  
middle-sized
```

If the selected species is neither the largest nor the smallest, the outer **else** branch should identify it as the middle-sized species group. For this dataset:

- Adelie is the largest.
- Gentoo is the middle-sized group.
- Chinstrap is the smallest.

11. Compare the Selected Count to the Average

Use a **conditional expression** to determine whether the selected species count is above or below the average number of observations per species. Create a variable named `average_position`. The statement should follow this form:

```
average_position = "above" if selected_count > average_penguins else "below"
```

For this dataset, none of the three species counts is exactly equal to the average, so the result will always be either above or below.

12. Display the Selected Species Statistics

For a valid species selection, display:

- the selected species,
- its number of observations,
- its percentage of the complete dataset,
- its size category,
- whether it is the largest, smallest, or middle-sized group,
- and whether its count is above or below the dataset average.

Continue using f-strings and the formatting conventions introduced in Module 3. The percentage should be displayed with two decimal places, and the average should be displayed with one decimal place. Your output might include statements similar to:

Gentoo has 123 observations, representing 35.96% of the dataset.

Gentoo is classified as a medium species group.

Gentoo is the middle-sized species group.

Its observation count is above the dataset average of 114.0.

Use the variables calculated by your program rather than placing these values directly in the output statements (in other words, do not “hardcode” the data into the output).

Expected Output

The first portion of your program should continue to produce the Module 3 statistics.

A run in which the user selects Gentoo should produce output similar to:

Introducing the Palmer Penguins:

Chinstrap!

Gentoo!

and last but not least...

Adelie!

There are a total of 3 penguin species in this dataset.

There are a total of 342 penguins in the dataset.

The average number of penguins per species is 114.0.

The average rounded up is 114 penguins.

Chinstrap: 68 (19.88%)

Gentoo: 123 (35.96%)

Adelie: 151 (44.15%)

The species with the most observations is Adelie with 151 penguins.

The species with the fewest observations is Chinstrap with 68 penguins.

Explore a species:

Enter Chinstrap, Gentoo, or Adelie: Gentoo

Gentoo has 123 observations, representing 35.96% of the dataset.

Gentoo is classified as a medium species group.

Gentoo is the middle-sized species group.

Its observation count is above the dataset average of 114.0.

A run with an invalid species should still display the original dataset statistics, but the final section should appear similar to:

Explore a species:

Enter Chinstrap, Gentoo, or Adelie: Emperor

Emperor is not one of the species in this dataset.

Minor whitespace differences are acceptable as long as the output is readable and contains all required information.

Test Your Branches

Before submitting the assignment, run your program several times.

At minimum, test:

- Chinstrap
- Gentoo
- Adelie
- one invalid value such as Emperor

These tests exercise the different branches in your program. Do not change the source code between tests. The program should make the correct decision based on the user's input.

Data Science Connection

Data analysis often involves more than calculating statistics. Programs also need to **interpret values and make decisions based on them**. In this assignment, our existing lists provide the data, while branches allow us to classify and interpret that data. For example, we use:

```
if
elif
else
```

to place percentages into categories, relational operators such as:

<
>=
==

to compare values, logical operators such as:

and
or

to combine conditions, and the membership operators **in** and **not in** to determine whether a value appears in a collection. These techniques will become increasingly useful as we begin making decisions based on larger datasets.

Readability

In addition to whether your program runs correctly, your assignments are graded on code readability. Keep your program readable by:

- using meaningful variable names,
- using consistent whitespace around operators,
- indenting branch statements consistently using four spaces,
- separating logical sections with blank lines,
- keeping lines reasonably short,
- adding comments where appropriate,
- formatting lists so their contents are easy to read,
- and following the Python naming conventions demonstrated in this assignment.

Add comments identifying the new logical sections of your program where appropriate, such as:

prompt the user to select a species

validate the species selection

determine the selected species

classify the selected species

determine the species position

output selected species statistics

Pay particular attention to indentation. Statements that belong to an **if**, **elif**, or **else** branch must be indented correctly.

Screenshot

Run the completed program in Google Colab. For your screenshot, use one of the three valid species names so the screenshot demonstrates that the new branching portion of the program executes successfully. The screenshot does not need to include all of your source code because the .py file will be reviewed separately. Save the screenshot as a .png or .jpg file. The suggested filename is:

PalmerPenguinsM4.png

Submission

Upload the following two files to your forked GitHub repository:

- PalmerPenguinsM4.py
- screenshot of the program executing successfully (.png or .jpg)

Submit the URL of your forked GitHub repository to the Canvas assignment. **Do not submit the Google Colab .ipynb notebook.**